

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA5115 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5 , BL6)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Code of Conduct:

I **M Nishanth (192511154)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Nishanth

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

1. Write pseudocode to simulate a simplified TLS handshake process, including client request, server authentication using a digital certificate, session key generation, and secure data exchange.
2. Develop pseudocode for a firewall rule engine that filters incoming packets based on IP address, port number, and protocol, and decides whether to allow or block traffic.
3. Write pseudocode to detect and classify malware behavior (virus, worm, or Trojan) based on propagation method, file modification patterns, and user interaction.
4. Design pseudocode for an Intrusion Detection System (IDS) that uses both signature-based and anomaly-based detection to identify suspicious network activity.
5. Create pseudocode for an Intrusion Prevention System (IPS) that monitors network traffic in real time and automatically blocks malicious requests while logging intrusion attempts.

SIMATS ENGINEERING

ASSESSMENT TOOL 2 PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Question 1: Simplified TLS Handshake Simulation

Problem Understanding — Inputs, Outputs, and Constraints

- Inputs: client's supported cipher suites and TLS version, server's digital certificate and private key, a trusted root CA store.
- Outputs: a shared, secret session key established on both sides, and a stream of encrypted application data.
- Constraints: the server's certificate must be validated against a trusted root before any secret material is exchanged, and both sides must independently derive the identical session key without ever transmitting it in the clear.

Algorithm Design Explanation

- The handshake is modeled as a sequential exchange (Hello -> Certificate -> Key Exchange -> Finished) with an explicit abort path at each validation point, since any single failed check must stop the handshake rather than continue with a partially trusted session.
- Session key derivation uses values contributed by both parties (client_random, server_random, pre_master_secret) so that neither side alone controls the final key, and a Finished-message integrity check confirms both sides agree on everything exchanged so far before data flows.

Pseudocode

```
FUNCTION TLS_HANDSHAKE(client, server, trusted_root_CAs):  
  
    // ---- Step 1: Client Hello ----  
    client_random <- GENERATE_RANDOM()  
    SEND ClientHello(client_random, client.supported_ciphers, tls_version) TO server  
  
    // ---- Step 2: Server Hello + Certificate ----  
    RECEIVE ClientHello FROM client  
    server_random <- GENERATE_RANDOM()  
    chosen_cipher <- SELECT_CIPHER(client.supported_ciphers, server.supported_ciphers)  
    SEND ServerHello(server_random, chosen_cipher) TO client  
    SEND Certificate(server.digital_certificate) TO client  
    SEND ServerHelloDone TO client  
  
    // ---- Step 3: Client authenticates the server's certificate ----  
    RECEIVE Certificate FROM server
```

```

IF VERIFY_CERTIFICATE_CHAIN(server.digital_certificate, trusted_root_CAs) == FALSE THEN
    ABORT_HANDSHAKE("Certificate chain invalid or untrusted")
    RETURN FAILURE
END IF
IF CHECK_HOSTNAME(server.digital_certificate, requested_hostname) == FALSE THEN
    ABORT_HANDSHAKE("Certificate hostname mismatch")
    RETURN FAILURE
END IF
IF IS_EXPIRED(server.digital_certificate) == TRUE THEN
    ABORT_HANDSHAKE("Certificate expired")
    RETURN FAILURE
END IF

// ---- Step 4: Session key generation ----
pre_master_secret <- GENERATE_RANDOM()
encrypted_pms <- ENCRYPT(pre_master_secret, server.public_key_from_certificate)
SEND ClientKeyExchange(encrypted_pms) TO server

RECEIVE ClientKeyExchange FROM client
pre_master_secret <- DECRYPT(encrypted_pms, server.private_key)

master_secret <- DERIVE_MASTER_SECRET(pre_master_secret, client_random, server_random)
session_key <- DERIVE_SESSION_KEY(master_secret)

// ---- Step 5: Finished messages confirm handshake integrity ----
SEND ChangeCipherSpec TO server
SEND Finished( ENCRYPT(HASH(all_handshake_messages), session_key) ) TO server

RECEIVE Finished FROM client
expected_hash <- HASH(all_handshake_messages_seen_by_server)
IF DECRYPT(received_finished_value, session_key) != expected_hash THEN
    ABORT_HANDSHAKE("Handshake integrity verification failed")
    RETURN FAILURE
END IF
SEND ChangeCipherSpec TO client
SEND Finished( ENCRYPT(HASH(all_handshake_messages), session_key) ) TO client

// ---- Step 6: Secure data exchange ----
WHILE session_active == TRUE DO
    IF client HAS pending_data THEN
        SEND ENCRYPT(pending_data, session_key) TO server
    END IF
    IF server HAS pending_data THEN
        SEND ENCRYPT(pending_data, session_key) TO client
    END IF
    IF EITHER side SENDS CloseNotify THEN
        session_active <- FALSE
    END IF
END WHILE

RETURN SUCCESS
END FUNCTION

```

Completeness — Cases Handled

- Handles the failure case of an untrusted/invalid certificate chain by aborting before any session key is derived, preventing a man-in-the-middle from proceeding with a fraudulent certificate.
- Handles hostname mismatch and certificate expiry as separate explicit checks, since a chain can be technically valid yet still wrong for this connection.

- Handles handshake-integrity failure at the Finished-message stage, catching tampering or a downgrade attempt that occurred earlier in the exchange.
- The data-exchange phase is modeled as a loop bounded by an explicit session-termination condition (CloseNotify), rather than assuming an unbounded or unterminated exchange.

Question 2: Firewall Rule Engine

Problem Understanding — Inputs, Outputs, and Constraints

- Inputs: an incoming packet (source IP, destination IP, destination port, protocol) and an ordered list of firewall rules, each with a source/destination IP pattern, port, protocol, action (ALLOW/BLOCK), and priority.
- Outputs: a per-packet decision (ALLOW or BLOCK) and a log entry recording that decision.
- Constraints: rules must be evaluated in priority order and the first matching rule must decide the packet's fate (first-match-wins), with an explicit default-deny fallback when no rule matches, since an undefined case must not silently become an implicit allow.

Algorithm Design Explanation

- Rules are pre-sorted once by priority (not re-sorted per packet) so that per-packet filtering only requires a single linear scan, keeping the hot path efficient for high packet rates.
- Matching is expressed as a conjunction (AND) of independent field checks (IP, port, protocol), each of which also supports an 'ANY' wildcard, so the same matching function cleanly handles both specific and broad rules without special-casing.

Pseudocode

```
STRUCTURE Rule:
    rule_id, src_ip, dst_ip, port, protocol, action, priority

FUNCTION MATCHES(packet, rule):
    ip_ok  <- (rule.src_ip == "ANY" OR packet.src_ip == rule.src_ip)
           AND (rule.dst_ip == "ANY" OR packet.dst_ip == rule.dst_ip)
    port_ok <- (rule.port == "ANY" OR packet.dst_port == rule.port)
    proto_ok <- (rule.protocol == "ANY" OR packet.protocol == rule.protocol)
    RETURN ip_ok AND port_ok AND proto_ok
END FUNCTION

FUNCTION FIREWALL_FILTER(packet, sorted_rule_list):
    // sorted_rule_list is pre-sorted by priority, descending, before this runs
    FOR EACH rule IN sorted_rule_list DO
```

```

        IF MATCHES(packet, rule) == TRUE THEN
            LOG_DECISION(packet, rule.action, rule.rule_id)
            RETURN rule.action
        END IF
    END FOR

    // No rule matched -> default-deny policy
    LOG_DECISION(packet, BLOCK, "default-deny")
    RETURN BLOCK
END FUNCTION

MAIN:
rule_list <- LOAD_RULES_FROM_CONFIG()
SORT rule_list BY priority DESCENDING
firewall_active <- TRUE

WHILE firewall_active == TRUE DO
    packet <- CAPTURE_NEXT_PACKET()
    IF packet == NULL THEN
        CONTINUE          // no packet ready this cycle
    END IF

    decision <- FIREWALL_FILTER(packet, rule_list)
    IF decision == ALLOW THEN
        FORWARD(packet)
    ELSE
        DROP(packet)
    END IF
END WHILE
END MAIN

```

Completeness — Cases Handled

- Handles the no-match case explicitly with a default-deny fallback, rather than leaving undefined traffic to fall through unblocked.
- Handles wildcard ('ANY') fields for IP, port, and protocol within the same matching function, so both broad and narrow rules are supported without duplicated logic.
- Handles an empty/no-packet capture cycle (packet == NULL) without breaking the monitoring loop, reflecting that real traffic arrives intermittently rather than continuously.
- Rule priority ordering ensures that when multiple rules could match the same packet (e.g., a broad BLOCK rule and a narrower ALLOW rule), the intended more-specific/higher-priority rule takes precedence deterministically.

Question 3: Malware Behavior Detection and Classification

Problem Understanding — Inputs, Outputs, and Constraints

- Inputs: observed behavior of a suspicious sample — its propagation method, its file modification pattern, and whether it requires user interaction to run/spread.
- Outputs: a classification label (VIRUS, WORM, TROJAN, or UNKNOWN) together with a confidence score for each category.
- Constraints: real malware samples do not always present a single unambiguous indicator, so the design must tolerate partial/overlapping evidence rather than requiring an exact match against one fixed rule.

Algorithm Design Explanation

- Rather than a rigid if/else decision tree (which breaks down when a sample shows mixed characteristics), a weighted scoring approach is used: each observed behavior adds points toward one or more categories, and the category with the highest accumulated score is selected — this is more robust to real-world ambiguity.
- A minimum confidence threshold is enforced before committing to a label, so a sample with weak or contradictory evidence is correctly routed to manual analysis instead of being forced into an incorrect category.

Pseudocode

```
FUNCTION CLASSIFY_MALWARE(sample):  
    virus_score <- 0  
    worm_score <- 0  
    trojan_score <- 0  
  
    // ---- Feature 1: propagation method ----  
    IF sample.propagation_method == "INFECTS_OTHER_FILES" THEN  
        virus_score <- virus_score + 3  
    ELSE IF sample.propagation_method == "SPREADS_AUTONOMOUSLY_OVER_NETWORK" THEN  
        worm_score <- worm_score + 3  
    ELSE IF sample.propagation_method == "DOES_NOT_SELF_PROPAGATE" THEN  
        trojan_score <- trojan_score + 3  
    END IF  
  
    // ---- Feature 2: file modification pattern ----  
    IF sample.modifies_or_attaches_to_existing_executables == TRUE THEN  
        virus_score <- virus_score + 2  
    END IF  
    IF samplecreates_independent_copies_of_itself == TRUE THEN  
        worm_score <- worm_score + 2  
    END IF  
    IF sample.disguised_as_legitimate_software == TRUE THEN  
        trojan_score <- trojan_score + 2
```

```

END IF

// ---- Feature 3: user interaction requirement ----
IF sample.requires_user_to_execute_it == TRUE THEN
    virus_score <- virus_score + 1
    trojan_score <- trojan_score + 1
ELSE
    worm_score <- worm_score + 1    // worms typically need no user action
END IF

scores <- { "VIRUS": virus_score, "WORM": worm_score, "TROJAN": trojan_score }
best_label <- KEY_WITH_MAX_VALUE(scores)

IF scores[best_label] < MINIMUM_CONFIDENCE_THRESHOLD THEN
    RETURN ("UNKNOWN - flagged for manual analysis", scores)
END IF

RETURN (best_label, scores)
END FUNCTION

```

Completeness — Cases Handled

- Handles the ambiguous case where a sample shows mixed indicators (e.g., some worm-like and some trojan-like behavior) by comparing accumulated scores rather than stopping at the first matching condition.
- Handles the low-evidence case via an explicit confidence threshold, avoiding a forced, potentially wrong classification when the available behavioral evidence is weak.
- Handles all three malware categories through the same generalized scoring loop and feature set rather than three separate hand-written decision paths, keeping the logic consistent and easy to extend with a fourth category later.

Question 4: Intrusion Detection System (Signature + Anomaly Based)

Problem Understanding — Inputs, Outputs, and Constraints

- Inputs: a live network traffic stream, a signature database of known attack patterns, and a baseline profile describing statistically normal traffic.
- Outputs: real-time alerts for detected intrusions (signature-based or anomaly-based), each with a severity level, and an updated baseline profile that adapts to legitimate traffic over time.
- Constraints: the system must catch both known attacks (fast, low false-positive signature matching) and previously unseen attacks (anomaly detection), since relying on only one method leaves a clear blind spot in either direction.

Algorithm Design Explanation

- Signature-based detection is checked first because it is typically fast and highly precise for known patterns; only if no signature matches does the system fall through to the more computationally expensive, probabilistic anomaly check — an efficiency-driven ordering.
- The baseline profile is continuously updated with traffic that passes both checks, so the anomaly detector adapts to legitimate changes in normal usage over time rather than remaining static and increasingly inaccurate.

Pseudocode

```
FUNCTION SIGNATURE_DETECT(packet, signature_db):
  FOR EACH signature IN signature_db DO
    IF PATTERN_MATCH(packet.payload, signature.pattern) == TRUE THEN
      RETURN { found: TRUE, attack_name: signature.name }
    END IF
  END FOR
  RETURN { found: FALSE }
END FUNCTION

FUNCTION ANOMALY_DETECT(packet, baseline_profile):
  feature_vector <- EXTRACT_FEATURES(packet) // e.g., size, rate, port, flags
  deviation <- CALCULATE_STATISTICAL_DEVIATION(feature_vector, baseline_profile)
  RETURN deviation
END FUNCTION

FUNCTION IDS_MONITOR(traffic_stream, signature_db, baseline_profile):
  monitoring_active <- TRUE

  WHILE monitoring_active == TRUE DO
    packet <- GET_NEXT_PACKET(traffic_stream)
    IF packet == NULL THEN
      CONTINUE
    END IF

    // ---- Signature-based check (fast path) ----
    sig_result <- SIGNATURE_DETECT(packet, signature_db)
    IF sig_result.found == TRUE THEN
      ALERT("Signature match: " + sig_result.attack_name, packet, SEVERITY_HIGH)
      LOG_INTRUSION(packet, sig_result.attack_name, "signature-based")
      CONTINUE
    END IF

    // ---- Anomaly-based check (fallback) ----
    anomaly_score <- ANOMALY_DETECT(packet, baseline_profile)
    IF anomaly_score > ANOMALY_THRESHOLD THEN
      ALERT("Anomalous traffic, score=" + anomaly_score, packet, SEVERITY_MEDIUM)
      LOG_INTRUSION(packet, "statistical anomaly", "anomaly-based")
    ELSE
      UPDATE_BASELINE(baseline_profile, packet) // reinforce normal-traffic model
    END IF
  END WHILE
END FUNCTION
```

Completeness — Cases Handled

- Handles the known-attack case via signature matching and the unknown/zero-day case via anomaly scoring, so the two methods cover each other's blind spots.

- Handles the empty-packet/idle-cycle case (packet == NULL) without terminating the monitoring loop.
- Handles baseline drift by updating the profile only with traffic that was not flagged, so the adaptive model is not poisoned by traffic that was itself anomalous or malicious.
- Assigns different severity levels to signature-based versus anomaly-based alerts, reflecting that a precise signature match generally warrants higher-confidence response than a statistical deviation alone.

Question 5: Intrusion Prevention System (Real-Time Blocking and Logging)

Problem Understanding — Inputs, Outputs, and Constraints

- Inputs: a live network traffic stream, a signature database, a baseline profile, and a dynamic blocklist of recently malicious source IPs.
- Outputs: real-time drop/forward decisions enforced automatically (not just alerts), an updated blocklist, and a persistent intrusion log.
- Constraints: unlike an IDS, the system must take action in real time (drop malicious traffic itself) rather than only alerting, while avoiding permanently blocking a source on a single borderline signal, and must eventually expire temporary blocks.

Algorithm Design Explanation

- Builds directly on the IDS detection logic (signature + anomaly) but adds an enforcement layer: a fast blocklist check runs first for already-known-bad sources (cheapest check, handles repeat offenders instantly), then combined detection determines whether to drop-and-block, forward-with-monitoring, or forward-and-learn.
- A time-bounded block (rather than a permanent one) combined with periodic expiry balances proactive prevention against the risk of permanently locking out a source based on a single incident, such as a misclassified legitimate burst of traffic.

Pseudocode

```
FUNCTION DETERMINE_REASON(sig_result, anomaly_score):
  IF sig_result.found == TRUE THEN
    RETURN "signature: " + sig_result.attack_name
  ELSE
    RETURN "anomaly score: " + anomaly_score
  END IF
END FUNCTION
```

```

FUNCTION LOG_INTRUSION(packet, reason, timestamp):
    entry <- { time: timestamp, src_ip: packet.src_ip, dst_ip: packet.dst_ip,
               port: packet.dst_port, reason: reason }
    APPEND entry TO intrusion_log_file
END FUNCTION

FUNCTION IPS_MONITOR(traffic_stream, signature_db, baseline_profile, block_list):
    monitoring_active <- TRUE

    WHILE monitoring_active == TRUE DO
        packet <- GET_NEXT_PACKET(traffic_stream)
        IF packet == NULL THEN
            CONTINUE
        END IF

        // ---- Fast path: already-known malicious source ----
        IF packet.src_ip IN block_list THEN
            DROP(packet)
            LOG_INTRUSION(packet, "blocklisted source", CURRENT_TIME())
            CONTINUE
        END IF

        // ---- Combined detection ----
        sig_result <- SIGNATURE_DETECT(packet, signature_db)
        anomaly_score <- ANOMALY_DETECT(packet, baseline_profile)

        IF sig_result.found == TRUE OR anomaly_score > CRITICAL_THRESHOLD THEN
            DROP(packet)
            ADD_TO_BLOCKLIST(block_list, packet.src_ip, BLOCK_DURATION)
            reason <- DETERMINE_REASON(sig_result, anomaly_score)
            LOG_INTRUSION(packet, reason, CURRENT_TIME())
            ALERT_ADMIN(packet, reason)
        ELSE IF anomaly_score > WARNING_THRESHOLD THEN
            FORWARD(packet)
            LOG_INTRUSION(packet, "suspicious - monitored, not blocked", CURRENT_TIME())
        ELSE
            FORWARD(packet)
            UPDATE_BASELINE(baseline_profile, packet)
        END IF

        // ---- Periodic maintenance ----
        IF TIME_SINCE_LAST_CLEANUP() > CLEANUP_INTERVAL THEN
            EXPIRE_OLD_BLOCKLIST_ENTRIES(block_list)
        END IF
    END WHILE
END FUNCTION

```

Completeness — Cases Handled

- Handles repeat offenders efficiently via a blocklist fast-path check before running the more expensive detection logic again.
- Handles three distinct confidence levels (critical/blocked, warning/monitored-only, normal/learn) rather than a single binary block-or-allow decision, reducing the chance that a moderately unusual but legitimate packet is dropped outright.
- Handles block-list expiry via periodic cleanup, ensuring a temporarily blocked source is not locked out indefinitely if the block was triggered by a transient issue.

- Every enforcement action (block, drop, forward-with-monitoring) is paired with a logged entry, satisfying the requirement to log intrusion attempts alongside automatic blocking.

Overall Conclusion

Each pseudocode design follows a consistent structure — explicit inputs/outputs, named functions with single responsibilities, and standard control constructs (IF/ELSE, WHILE, FOR EACH) — so the logic reads clearly independent of any specific programming language. Across all five tasks, particular attention was given to the failure and edge cases that a real security system must not ignore: certificate/handshake failures in TLS, unmatched traffic in the firewall, ambiguous or low-confidence samples in malware classification, zero-day attacks that evade signatures in the IDS, and transient false positives in the IPS's automatic blocking. Together these five designs form a coherent security stack — from establishing a trusted channel (TLS), to filtering traffic at the perimeter (firewall), to classifying and detecting threats (malware classifier, IDS), to actively stopping them in real time (IPS) — directly addressing CO5's focus on viruses, worms, firewall architecture, and IDS/IPS functionality.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient